

Nästlade klasser

Vi har hittills använt våra objekt i väldigt enkla omgivningar. Vi har inte alls använt oss av *komposition av objekt* (kallas även nästlade klasser). Det innebär att en klass innehåller objekt av samma eller andra klasser som datamedlemmar. Man kan i C++ nästla objekt i princip hur djupt som helst. I större program har man ofta många nivåer av nästlade klasser. Vi skall nu visa hur vi kan nästla in vår `Coordinate`-klass i andra klasser. Som exempel skall vi ta en cirkel som vi definierar som en mittpunkt och en radie. Mittpunkten är ju logiskt att skriva som en koordinat, varvid vi kommer till följande simplifierade cirkel:

```
class Circle {
public:
    // konstruktörer
    Circle (const Coordinate & Origin, float Radius);

    // accessera medlemmar
    const Coordinate & origin () const;
    float radius () const;

    // ändra radie och position
    void setOrigin (const Coordinate & Origin);
    void setRadius (float Radius);

private:
    // radie och position
    Coordinate m-Origin;
    float m_Radius;
}
```

Här har vi nu en medlem `m-Origin` som är av typen `Coordinate`. Det är alltså inget speciellt med att använda klasser i klasser, så länge som alla använda klasserna är definierade då de används. Notera att ovan har vi ingen *kod* överhuvudtaget för våra metoder. Vi skall nu visa hur den kan definieras separat. Det är det normala sättet att skapa klasser i C++. Java-syntaxen att placera koden direkt efter metoddefinitionen används vanligen endast för små och korta metoder (som dem vi hittills definierat). Klassens egentliga kod definieras nu på följande sätt:

```
Circle::Circle (const Coordinate & Origin, float Radius) {
    m-Origin = Origin;
    m_Radius = Radius;
}

const Coordinate & Circle::origin () const {
    return m-Origin;
}

float Circle::radius () const {
    return m_Radius;
}

void Circle::setOrigin (const Coordinate & Origin) {
    m-Origin = Origin;
}

void Circle::setRadius (float Radius) {
```

```
m_Radius = Radius;  
}
```

Vad som kännetecknar detta sätt att skriva metoderna är att man har ett *klassnamn::* framför varje metodnamn för att berätta för kompilatorn till vilken klass denna metod tillhör. Lämna man bort det prefixet är det frågan om en helt normal funktionsdefinition. På detta sätt kan vi enkelt separera själv klassdefinitionen från implementationen. Klassdefinitionen sätts normalt i en *headerfil* medan implementationen sätts i en *.cpp*-fil. Vi kunde således skapa två filer *Vector.h* som innehåller själva *class*-definitionen och *Vector.cpp* som innehåller implementationen av metoderna. Se [avsnittet Använda multipla filer i Kapitel 13](#) för mera information om denna modell.

För att bygga ut vårt bibliotek av figurer kunde vi tänka oss geometriska figurer såsom t.ex. fyrkanter, trianglar, linjer och punkter. Vi kommer att återkomma till dessa senare.

Klasser i klasser

Ett annat sätt att se på nästlade klasser är via klasser i klasser. Man kan definiera *lokala klasser* som kan användas inom den klass där den är definierad. Beroende på om klassen definieras i *public:* eller *private:* kan den användas även av externa entiteter.

Ett exempel på interna klasser ges i [avsnittet Exempel på dynamisk minneshantering i Kapitel 17](#), men det kräver mera kunskaper än vi har nu, bland annat om dynamisk minneshantering.

[Förutgående](#)
Konstruktor och destruktör

[Hem](#)
[Upp](#)

[Nästa](#)
Ärvning